Gates Foundation · NQPR Pipeline · Technical Reference

LangGraph Pipeline Architecture Report

prisma-langgraphed — Node Map, Data Flow, and State Evolution
Projectprisma-langgraphed
PipelineNQPR — Quarterly Portfolio Review
DateMay 2026

Total nodes5 top-level · 13 analyze · 21 causal · 6 research
Fan-out patterns9 Send() router–worker pairs
State fields328 WorkflowState · 62 AnalyzeState
Generated automatically from ARCHITECTURE.md and codebase analysis · prisma-langgraphed/docs/pipeline_architecture.pdf

Table of Contents

1. Overview and Key Patterns

prisma-langgraphed — LangGraph-native rewrite of the NQPR pipeline

The NQPR pipeline is a **multi-level LangGraph graph** that ingests a Gates Foundation program's investment collection and produces a quarterly portfolio review report with per-scope narratives, causal analysis, evidence packs, and projected strategic decisions. The pipeline is designed to run unattended with full checkpointing, resumability, and observability.

Graph Hierarchy

Graph	Nodes	Fan-out patterns	Key output
Top-level workflow	5	None (sequential)	Final report PDF + excerpts CSV
Analyze subgraph	13	3× Send() routers	scope_outputs, final_report_md
Causal pipeline subgraph	21	6× Send() routers	evidence_packs, link_assessments, decisions

Graph	Nodes	Fan-out patterns	Key output
Research dispatch subgraph	6	1× Send() to 4 workers	research_results
SLR / LBD / DeepWeb / Edison	5–7 each	Agent tool loops	papers, synthesis

Key Design Patterns

Send() fan-out — parallel workers per investment/scope/link
Idempotent nodes — early-return if state already populated
Reducer fields — operator.add / merge_scope_outputs accumulate across workers
Dispatch nodes — pure routers, return list[Send] only, no state writes
All inter-node data flows through TypedDict state — no globals
Async policy: Every node is `async def`. All blocking I/O uses `asyncio.to_thread()`. All LLM calls go through `acall_llm()`. No `ThreadPoolExecutor` — parallelism is exclusively via LangGraph `Send()`. Every `asyncio.*` call must have a `# asyncio-APPROVED-N` comment.

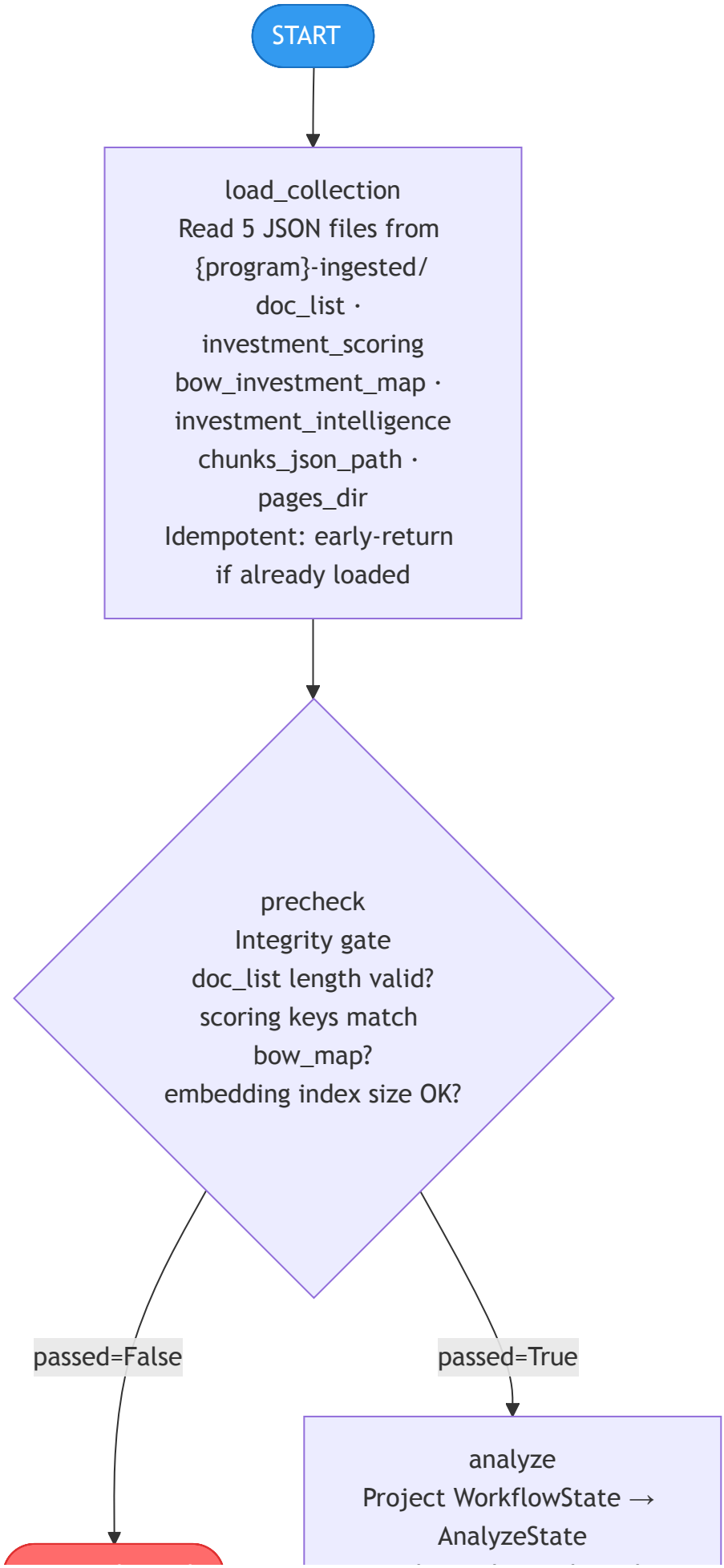
State Architecture

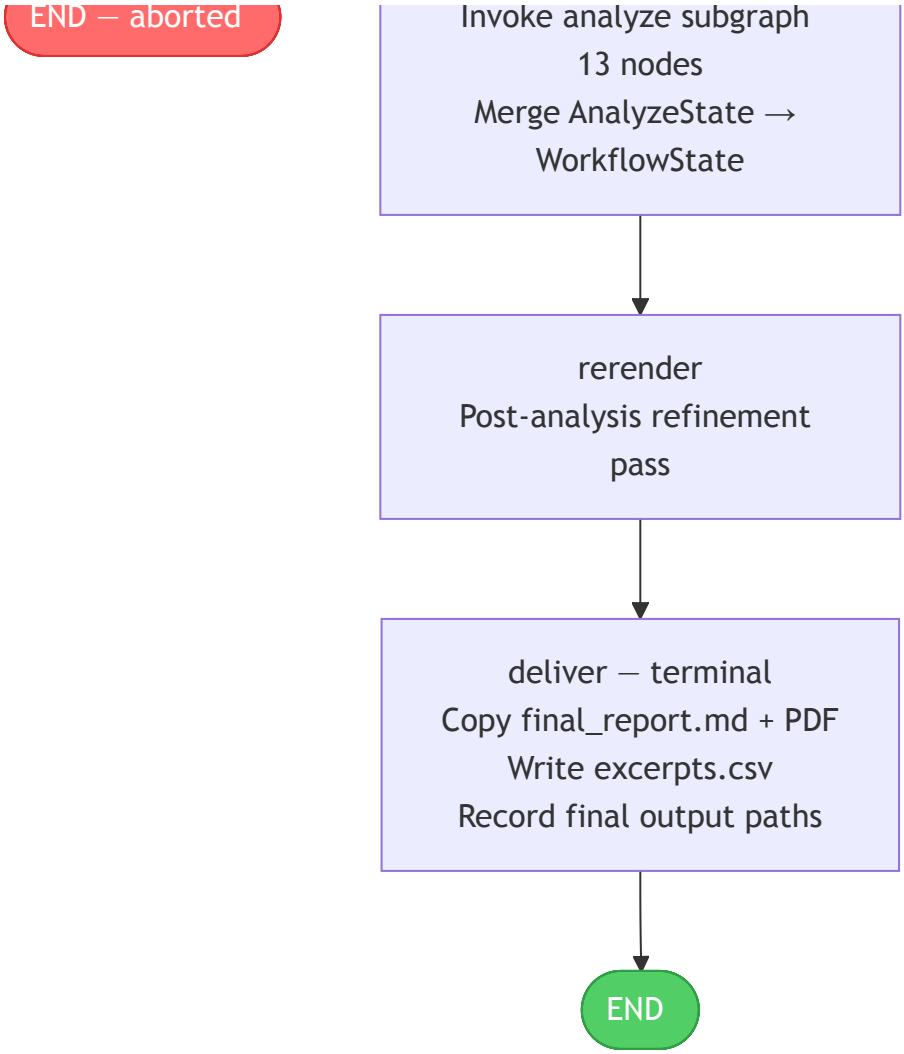
There are three primary `TypedDict` states plus eight Send() sub-states. The top-level `WorkflowState` (328 fields) is the source of truth. The `AnalyzeState` (62 fields) and `CausalState` (22 fields) are projected from it for their respective subgraphs, then merged back.

State type	Fields	Key reducers
`WorkflowState`	328	`_take_update` for fan-out accumulators
`AnalyzeState`	62	`merge_scope_outputs` for scope dict deep-merge
`CausalState`	22	`operator.add` for evidence/link/science lists
8× Send() sub-states	4–10 each	Single-item lists returned per worker

2. Top-Level Workflow

5 nodes · sequential · no fan-out · human interrupt before analyze (optional)
Figure 1 — Top-Level Workflow Graph


START
load_collection
Read 5 JSON files from {program}-ingested/
doc_list · investment_scoring
bow_investment_map · investment_intelligence
chunks_json_path · pages_dir
Idempotent: early-return if already loaded
precheck
Integrity gate
doc_list length valid?
scoring keys match bow_map?
embedding index size OK?
passed=False
passed=True
analyze
Project WorkflowState → AnalyzeState



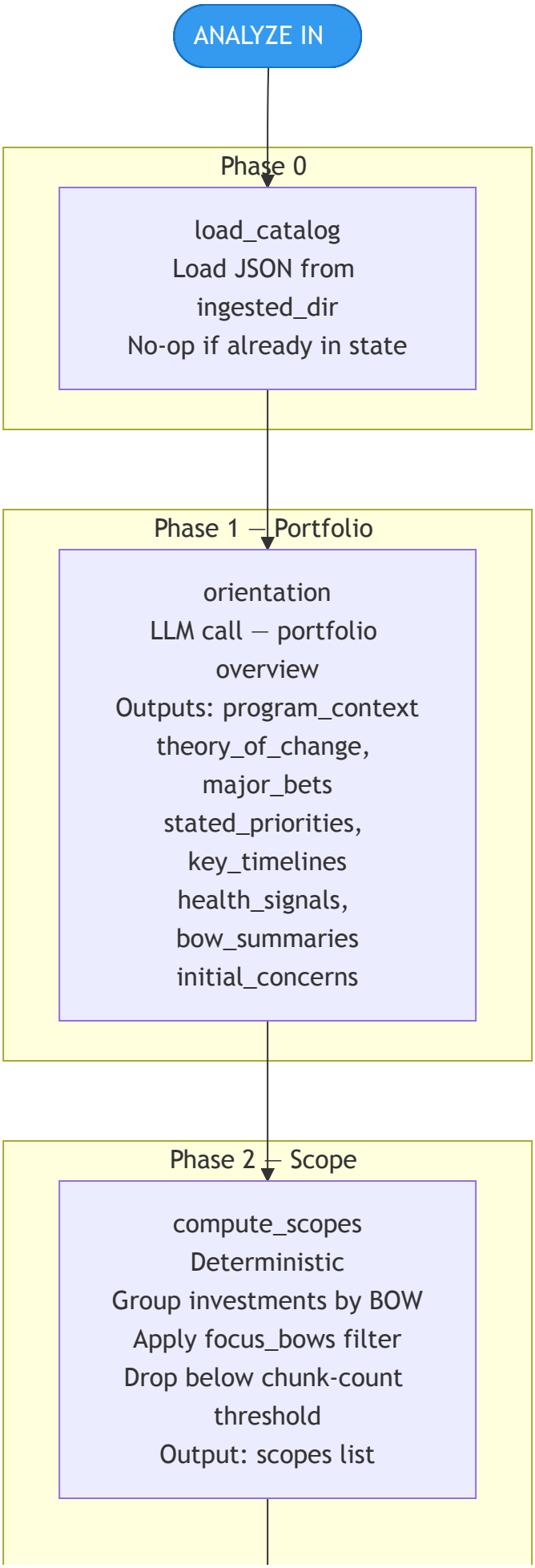
Node Operations

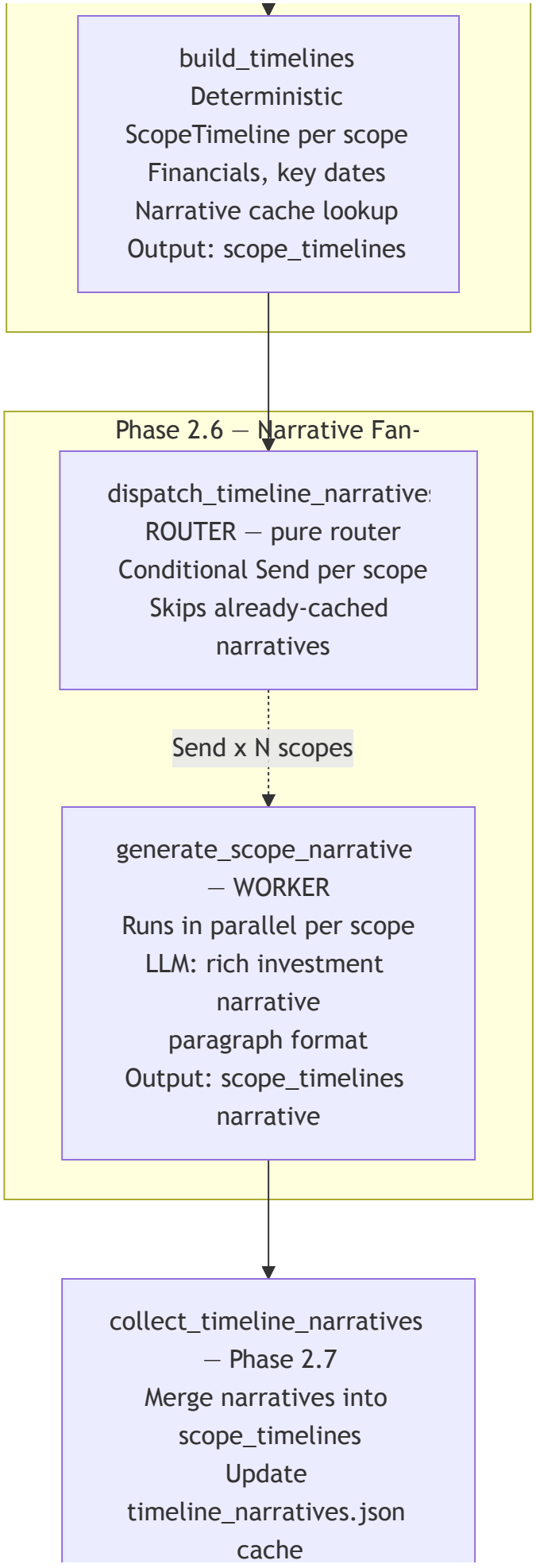
Node	Type	Operation	Key outputs added
`load_collection`	I/O	Reads 5 JSON files from `{{program}}-ingested/`. Idempotent: returns early if `doc_list` already populated in state.	`doc_list`, `investment_scoring`, `bow_investment_map`, `investment_intelligence`, `chunks_json_path`, `pages_dir`
`precheck`	Gate	Validates `doc_list` length ≥ 1, investment_scoring keys match bow_map, embedding index size consistent. Returns `precheck_passed=False` to terminate early on failure.	`precheck_passed`, `precheck_report`
`analyze`	Subgraph	Projects `WorkflowState → AnalyzeState`, invokes the 13-node analyze subgraph, merges all results back into `WorkflowState`. Optional human interrupt point (controlled by `--interactive` flag or `CHECKPOINT_HUMAN_INTERRUPTS` env var).	`scope_outputs`, `analyst_report`, `final_report_md`, `bibliography`, `coverage_pct`, `grade`, all tool traces
`rerender`	Refinement	Post-analysis report refinement pass. Currently a placeholder — passes state through unchanged. Reserved for future report polish passes.	`final_report_md` (optionally updated)

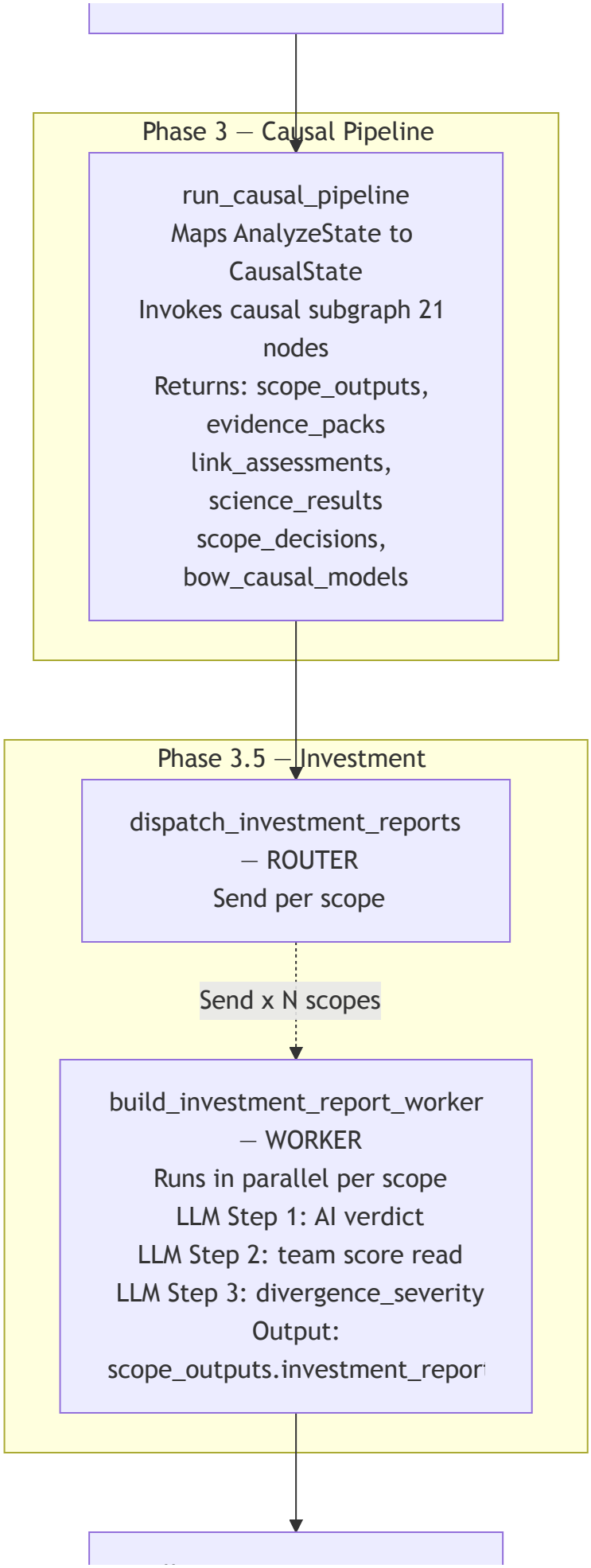
Node	Type	Operation	Key outputs added
`deliver`	Terminal	Copies `final_report.md` and PDF to the delivery directory. Writes `excerpts.csv` with evidence provenance. Records all final output paths in state.	`excerpts_csv_path`, `final_report_pdf_path`, `current_stage="delivered"`

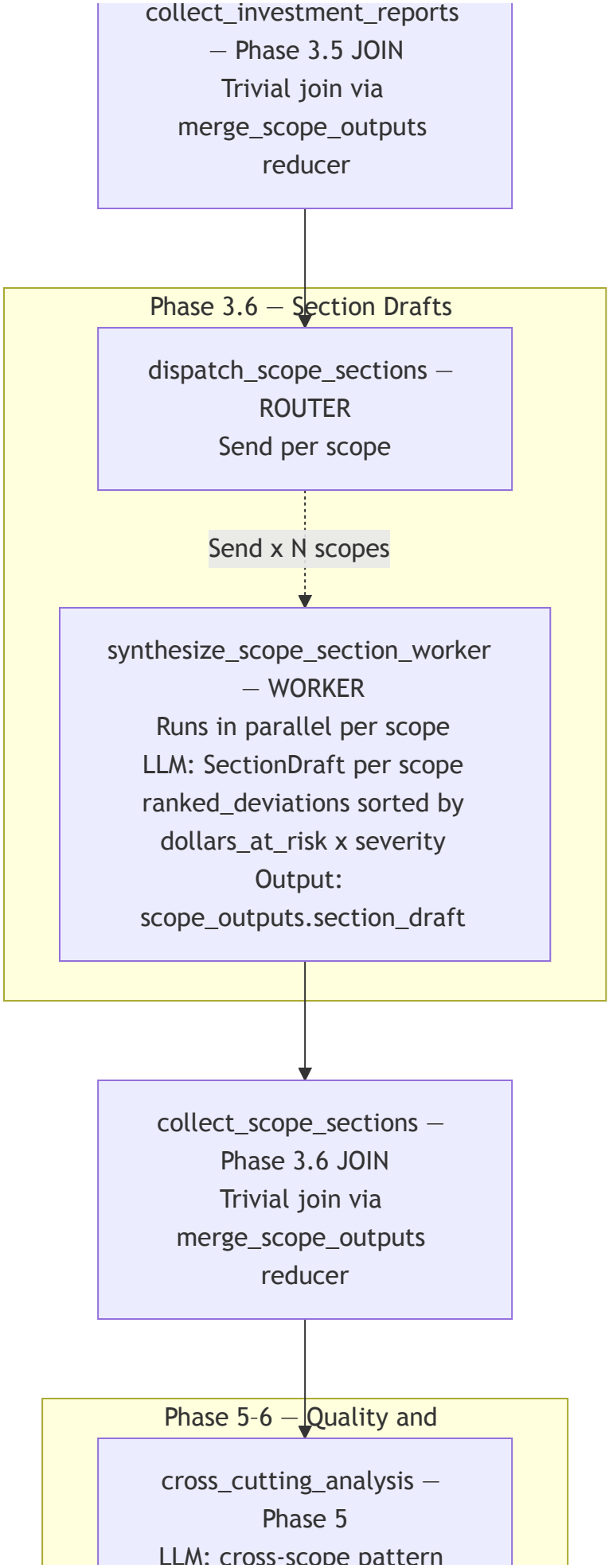
3. Analyze Subgraph

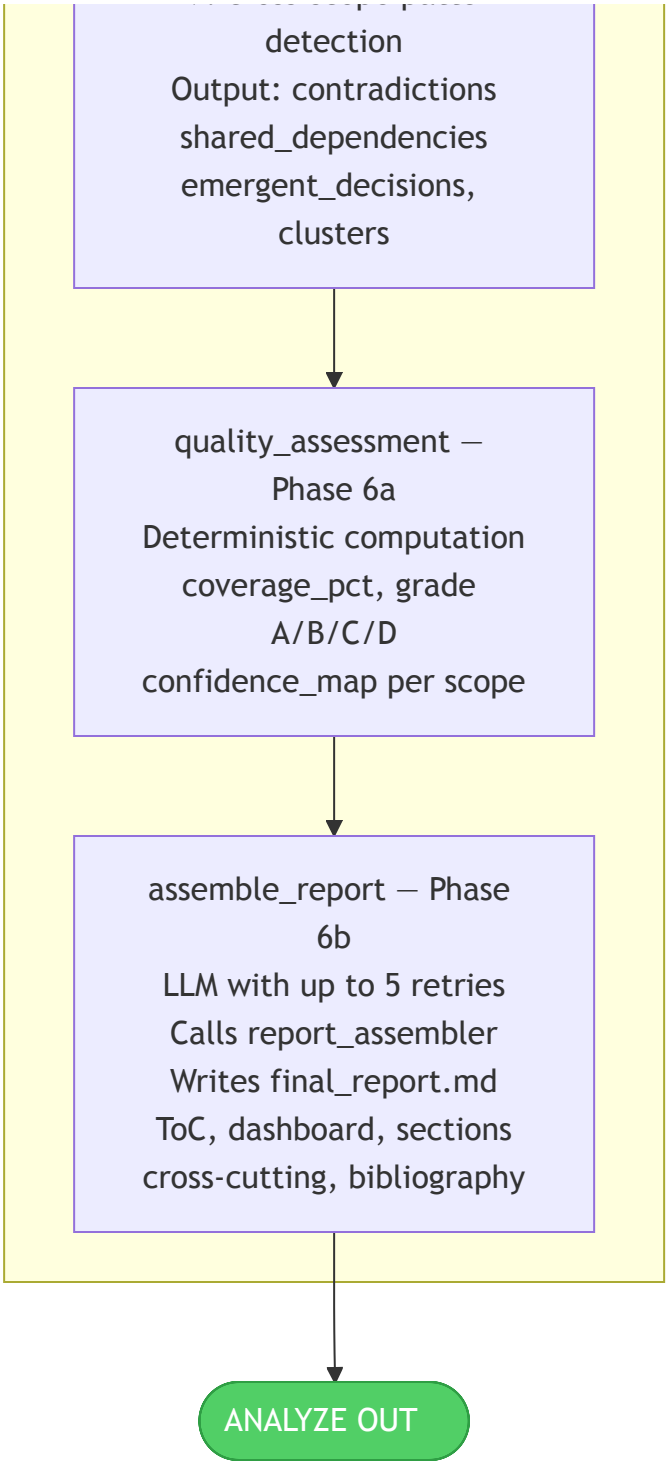
13 nodes · Phases 0–6b · 3× Send() fan-outs
Figure 2 — Analyze Subgraph (Phases 0–6b)


ANALYZE IN
Phase 0
load_catalog
Load JSON from ingested_dir
No-op if already in state
Phase 1 — Portfolio
orientation
LLM call — portfolio overview
Outputs: program_context theory_of_change, major_bets stated_priorities, key_timelines health_signals, bow_summaries initial_concerns
Phase 2 — Scope
compute_scopes
Deterministic
Group investments by BOW
Apply focus_bows filter
Drop below chunk-count threshold
Output: scopes list


build_timelines
Deterministic
ScopeTimeline per scope
Financials, key dates
Narrative cache lookup
Output: scope_timelines
Phase 2.6 — Narrative Fan-
dispatch_timeline_narrative
ROUTER — pure router
Conditional Send per scope
Skips already-cached
narratives
Send x N scopes
generate_scope_narrative
— WORKER
Runs in parallel per scope
LLM: rich investment
narrative
paragraph format
Output: scope_timelines
narrative
collect_timeline_narratives
— Phase 2.7
Merge narratives into
scope_timelines
Update
timeline_narratives.json
cache


Phase 3 — Causal Pipeline
run_causal_pipeline
Maps AnalyzeState to CausalState
Invokes causal subgraph 21 nodes
Returns: scope_outputs, evidence_packs
link_assessments, science_results
scope_decisions, bow_causal_models
Phase 3.5 — Investment
dispatch_investment_reports — ROUTER
Send per scope
Send x N scopes
build_investment_report_worker — WORKER
Runs in parallel per scope
LLM Step 1: AI verdict
LLM Step 2: team score read
LLM Step 3: divergence_severity
Output: scope_outputs.investment_repor


collect_investment_reports — Phase 3.5 JOIN
Trivial join via merge_scope_outputs reducer
Phase 3.6 — Section Drafts
dispatch_scope_sections — ROUTER
Send per scope
Send x N scopes
synthesize_scope_section_worker — WORKER
Runs in parallel per scope
LLM: SectionDraft per scope
ranked_deviations sorted by dollars_at_risk x severity
Output: scope_outputs.section_draft
collect_scope_sections — Phase 3.6 JOIN
Trivial join via merge_scope_outputs reducer
Phase 5-6 — Quality and
cross_cutting_analysis — Phase 5
LLM: cross-scope pattern



Phase-by-Phase Node Operations

Phase 0 — Catalog Load

Node	Operation	Outputs
`load_catalog`	Load pre-computed JSON from `ingested_dir`. No-op if collection data already in `AnalyzeState`. Enables standalone subgraph execution in tests.	Collection fields populated in `AnalyzeState`

Phase 1 — Portfolio Orientation

Node	Operation	Outputs
`orientation`	Single LLM call over all BOW summaries and document list. Produces a portfolio overview covering theory of change, major strategic bets, stated priorities, timeline-sensitive investments, portfolio health signals, per-BOW summaries, and initial concerns. Uses `analysis_model`.	`program_context` dict with 7 fields

Phase 2 — Scope Construction

Node	Operation	Outputs
`compute_scopes`	Deterministic grouping of investments by BOW. Applies `focus_bows` filter if set. Drops scopes whose total chunk count is below threshold. Pure computation, no LLM.	`scopes: list[ScopeSpec]`
`build_timelines`	Constructs a `ScopeTimeline` per scope from investment financial data and key dates. Checks narrative cache on disk. No LLM call here.	`scope_timelines: dict[scope_id, ScopeTimeline]`

Phase 2.6–2.7 — Narrative Fan-out

Node	Operation	Outputs
`dispatch_timeline_narratives`	Pure router. Emits a `Send()` for each scope whose narrative is not already cached. Returns `list[Send]` — writes nothing to state.	Routes to workers
`generate_scope_narrative`	Parallel worker (one per scope). LLM call producing a rich paragraph-format narrative for each investment. Uses `fast_model`.	`scope_timelines[scope_id].narrative`
`collect_timeline_narratives`	Joins all narrative results. Updates the `timeline_narratives.json` disk cache for future resumability.	`scope_timelines` with all narratives filled

Phase 3 — Causal Pipeline (delegate)

Node	Operation	Outputs
`run_causal_pipeline`	Maps `AnalyzeState → CausalState`, invokes the 21-node causal subgraph (see Section 4), maps results back. This is the most compute-intensive stage — it runs evidence collection, causal modeling, multi-turn investigation, and decision projection.	`scope_outputs` deeply populated, `bow_causal_models`, all trace accumulators

Phases 3.5–3.6 — Report Scaffolding Fan-outs

Node	Operation	Outputs
`dispatch_investment_reports`	Pure router. Sends one `Send()` per scope.	Routes to workers

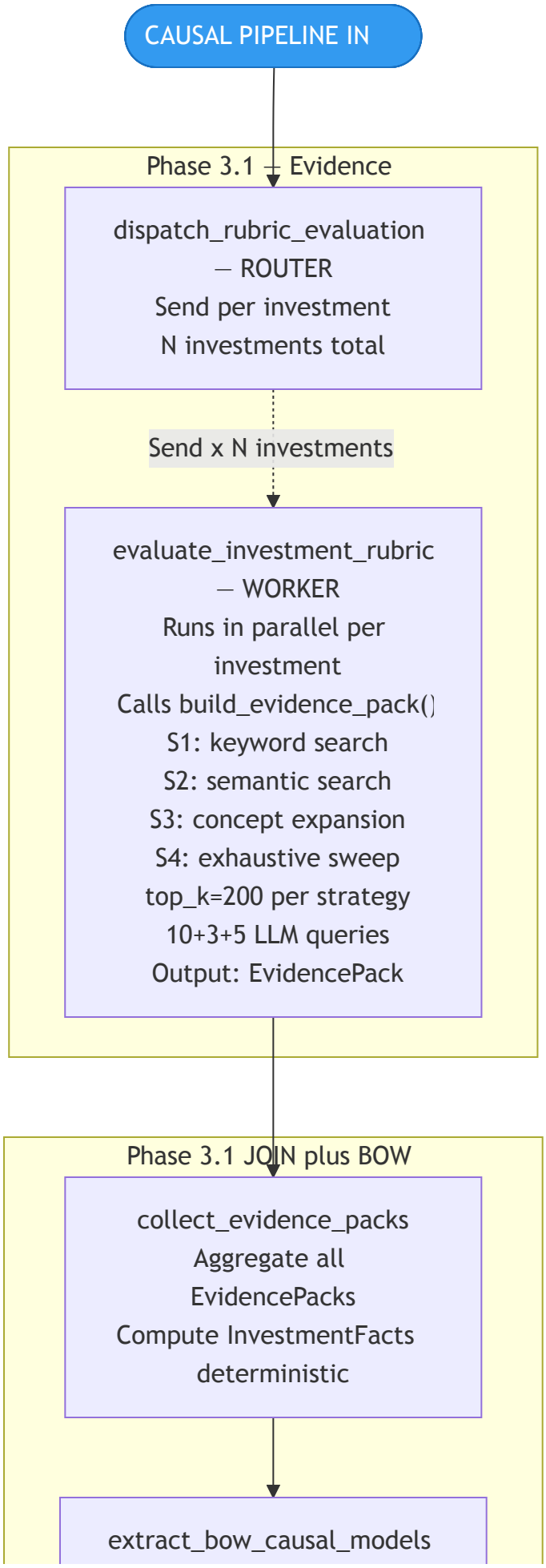
Node	Operation	Outputs
`build_investment_report_worker`	Three sequential LLM calls per scope: (1) AI verdict on investment performance, (2) read team scores from evidence, (3) compute divergence_severity score.	`scope_outputs[scope_id].investment_report`
`dispatch_scope_sections`	Pure router. Sends one `Send()` per scope.	Routes to workers
`synthesize_scope_section_worker`	LLM call producing a `SectionDraft` per scope. Includes ranked deviations sorted by `dollars_at_risk × severity` and narrative prose.	`scope_outputs[scope_id].section_draft`

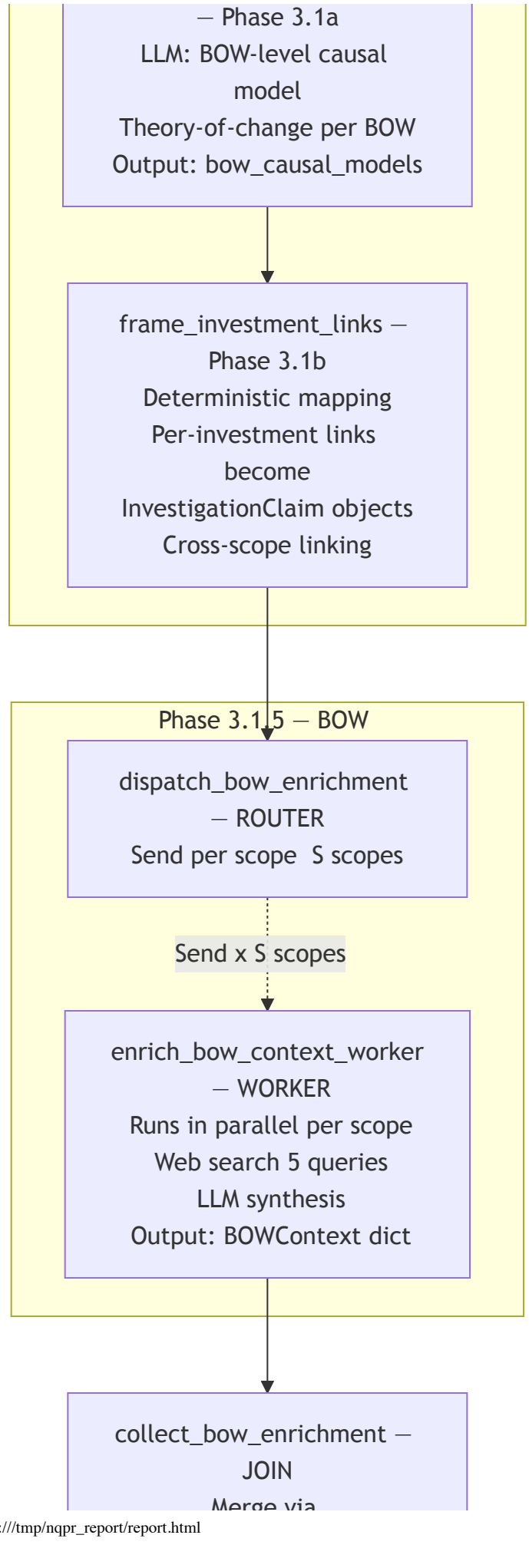
Phases 5–6 — Quality Assessment and Assembly

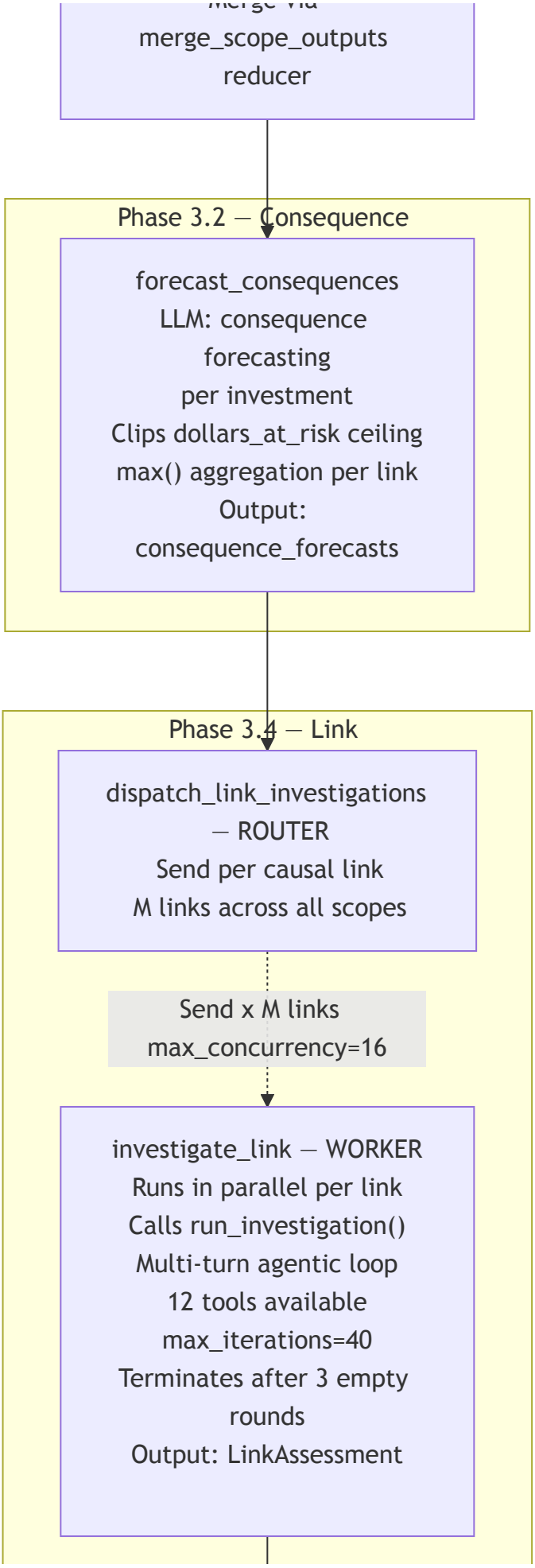
Node	Operation	Outputs
`cross_cutting_analysis`	LLM call over all scope outputs to detect portfolio-wide patterns: contradictions between scopes, shared dependencies, emergent decisions, thematic clusters. Uses `synthesis_model`.	`cross_cutting_analysis` dict
`quality_assessment`	Deterministic computation. Calculates `coverage_pct` (% of investments with sufficient evidence), assigns letter grade (A/B/C/D), builds `confidence_map[scope_id → float]`.	`coverage_pct`, `grade`, `confidence_map`
`assemble_report`	LLM call with up to 5 retry attempts. Calls `report_assembler.assemble_report()` to produce the final markdown report with table of contents, portfolio dashboard, per-scope sections, cross-cutting analysis, and deduplicated bibliography. Writes `final_report.md` to disk.	`final_report_md`, `final_report_md_path`, `bibliography`

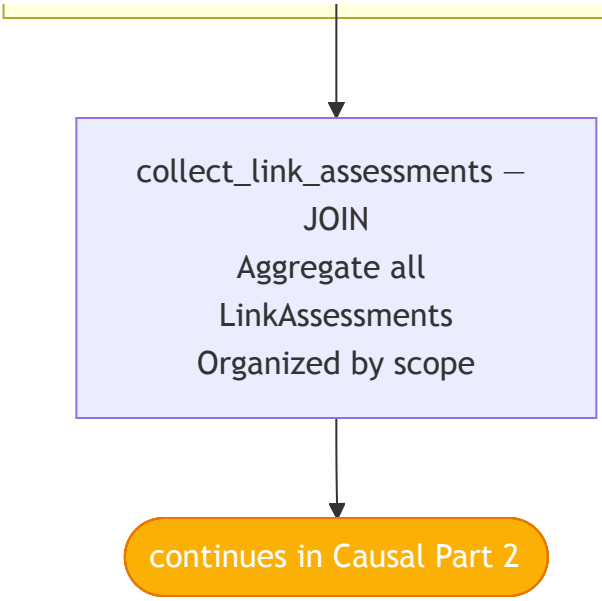
4. Causal Pipeline Subgraph — Part 1

Phases 3.1–3.4 · Evidence collection, BOW enrichment, consequence forecasting, link investigation
Figure 3 — Causal Pipeline: Phases 3.1–3.4 (Evidence → Link Investigation)


CAUSAL PIPELINE IN
Phase 3.1 — Evidence
dispatch_rubric_evaluation
— ROUTER
Send per investment
N investments total
Send x N investments
evaluate_investment_rubric
— WORKER
Runs in parallel per
investment
Calls build_evidence_pack()
S1: keyword search
S2: semantic search
S3: concept expansion
S4: exhaustive sweep
top_k=200 per strategy
10+3+5 LLM queries
Output: EvidencePack
Phase 3.1 JOIN plus BOW
collect_evidence_packs
Aggregate all
EvidencePacks
Compute InvestmentFacts
deterministic
extract_bow_causal_models


— Phase 3.1a
LLM: BOW-level causal model
Theory-of-change per BOW
Output: bow_causal_models
frame_investment_links — Phase 3.1b
Deterministic mapping
Per-investment links become InvestigationClaim objects
Cross-scope linking
Phase 3.1.5 — BOW
dispatch_bow_enrichment — ROUTER
Send per scope S scopes
Send x S scopes
enrich_bow_context_worker — WORKER
Runs in parallel per scope
Web search 5 queries
LLM synthesis
Output: BOWContext dict
collect_bow_enrichment — JOIN
Merge via


merge_scope_outputs
reducer
Phase 3.2 — Consequence
forecast_consequences
LLM: consequence
forecasting
per investment
Clips dollars_at_risk ceiling
max() aggregation per link
Output:
consequence_forecasts
Phase 3.4 — Link
dispatch_link_investigations
— ROUTER
Send per causal link
M links across all scopes
Send x M links
max_concurrency=16
investigate_link — WORKER
Runs in parallel per link
Calls run_investigation()
Multi-turn agentic loop
12 tools available
max_iterations=40
Terminates after 3 empty
rounds
Output: LinkAssessment



Node Operations — Phases 3.1–3.4

Phase 3.1 — Evidence Collection

Node	Operation
`dispatch_rubric_evaluation`	Pure router. Emits one `Send()` per investment with its `InvestmentRubricState`.
`evaluate_investment_rubric`	Worker (parallel per investment). Calls `rubric_evaluator.build_evidence_pack()` with four search strategies: S1 keyword, S2 semantic, S3 concept expansion, S4 exhaustive sweep. `top_k=200` per strategy. Issues 10+3+5 LLM queries. Produces an `EvidencePack` containing raw excerpts, scored claims, and source attribution.
`collect_evidence_packs`	Aggregates all `EvidencePack` objects. Deterministically computes `InvestmentFacts` (structured financial and programmatic facts) from the evidence.
`extract_bow_causal_models`	LLM call per BOW. Extracts a structured causal model (theory-of-change, key mechanisms, critical assumptions) for each BOW from the evidence packs. Populates `bow_causal_models`.
`frame_investment_links`	Deterministic mapping. Converts per-investment causal links into `InvestigationClaim` objects that span scopes and are ready for the investigation loop.

Phase 3.1.5 — BOW Context Enrichment

Node	Operation
`dispatch_bow_enrichment`	Pure router. Sends one `Send()` per scope.
`enrich_bow_context_worker`	Worker (parallel per scope). Issues 5 targeted web searches for external BOW context (sector benchmarks, global trends, comparable programs). LLM synthesizes results into a `BOWContext` dict.
`collect_bow_enrichment`	Trivial join via `merge_scope_outputs` reducer.

Phase 3.2 — Consequence Forecasting

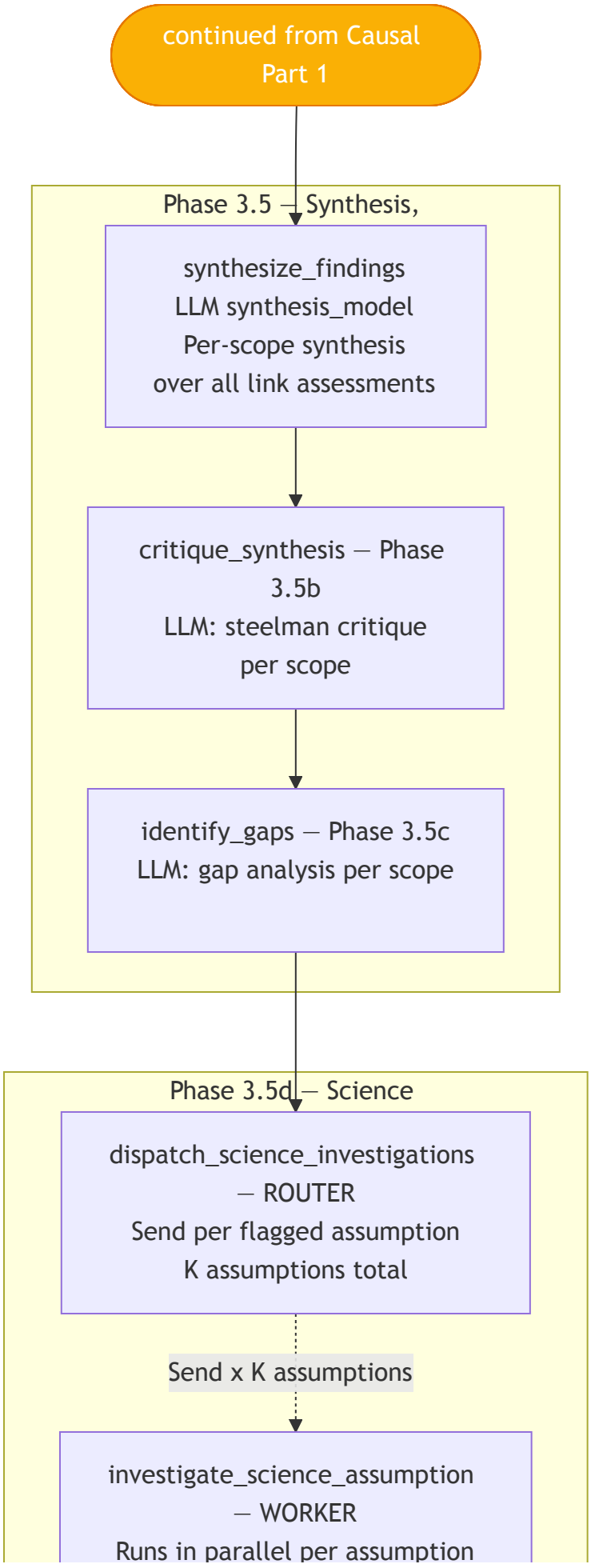
Node	Operation
`forecast_consequences`	LLM call per investment. Forecasts downstream consequences of each causal link failing or succeeding. Clips `dollars_at_risk` to a ceiling. Applies `max()` aggregation so the highest-risk estimate per link is kept.

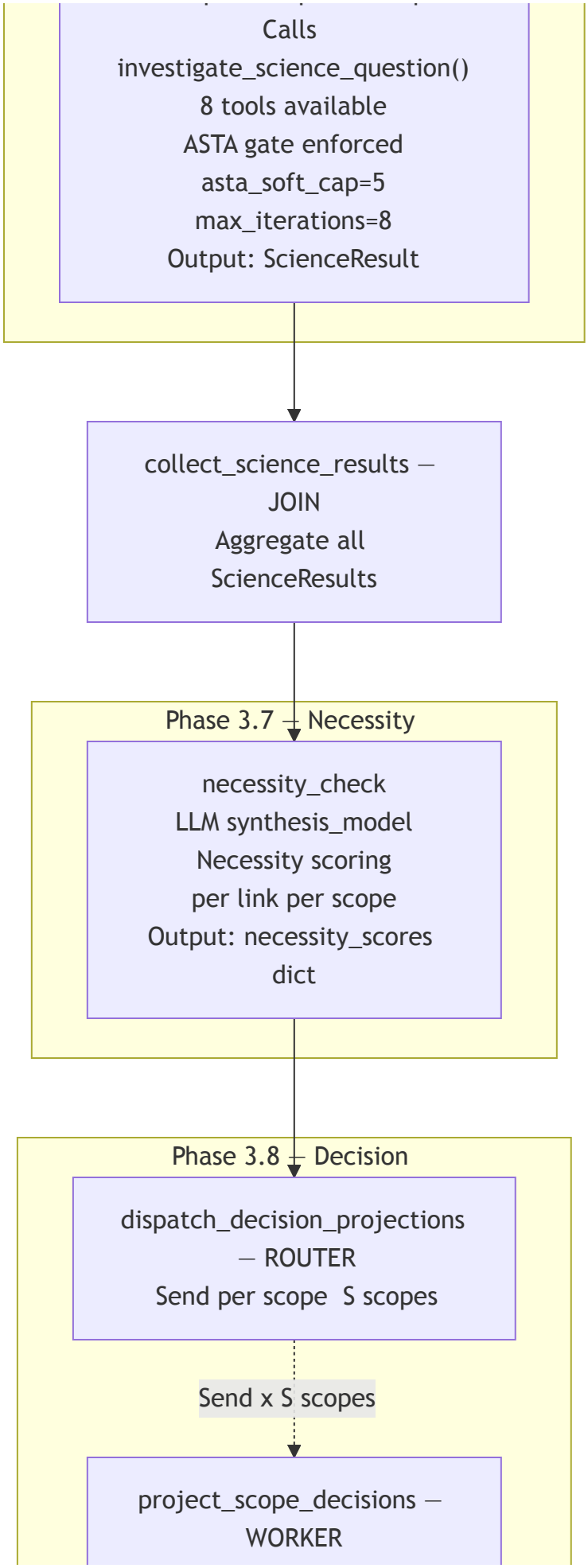
Phase 3.4 — Link Investigation

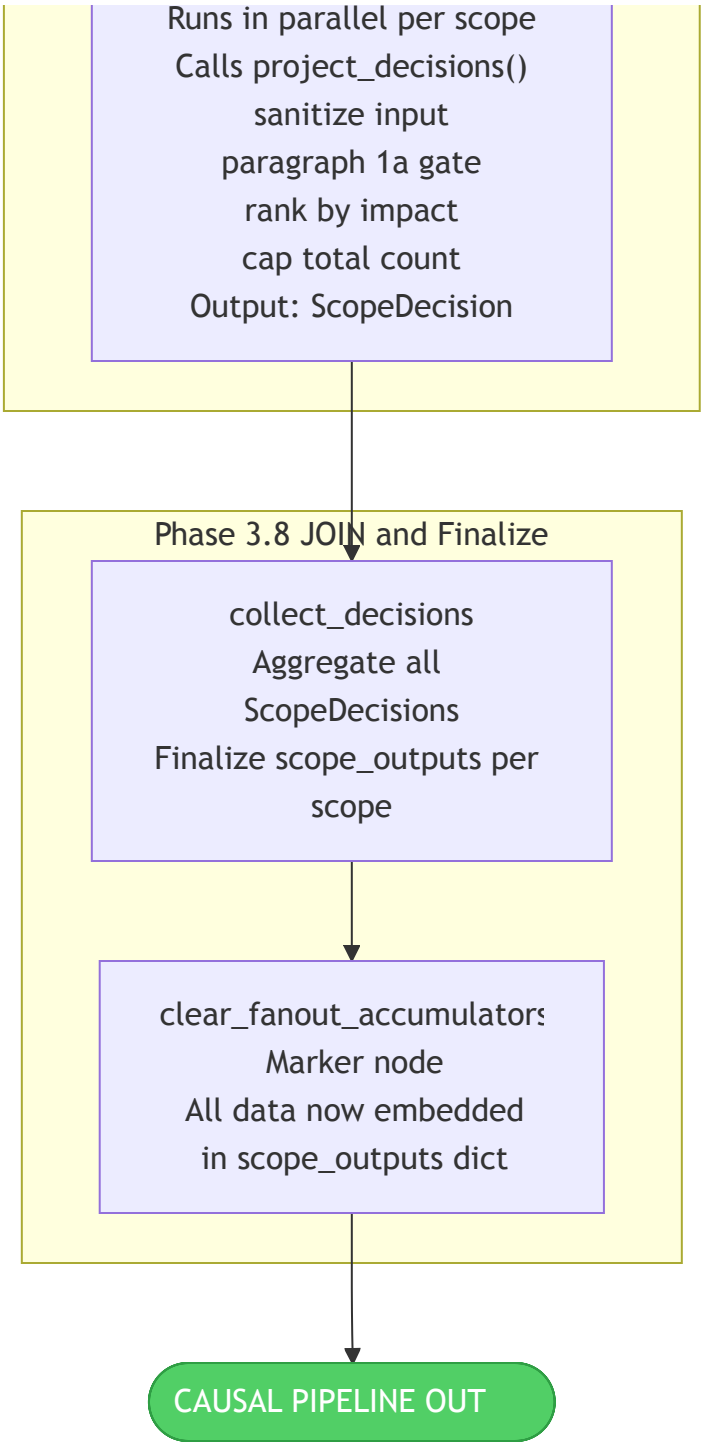
Node	Operation
`dispatch_link_investigations`	Pure router. Sends one `Send()` per causal link across all scopes (M links total).
`investigate_link`	Worker (parallel, `max_concurrency=16`). Calls `run_investigation()` — a multi-turn agentic loop with 12 tools available (6 search variants + document reading + compute). Runs up to 40 iterations. Terminates after 3 consecutive rounds with no new evidence. Produces a `LinkAssessment` with confidence score, evidence citations, and recommendation.
`collect_link_assessments`	Aggregates all `LinkAssessment` objects into per-scope lists via `operator.add` reducer.

4. Causal Pipeline Subgraph — Part 2

Phases 3.5–3.8 · Synthesis, science investigation, necessity check, decision projection
Figure 4 — Causal Pipeline: Phases 3.5–3.8 (Synthesis → Decision Projection)


continued from Causal Part 1
Phase 3.5 — Synthesis,
synthesize_findings
LLM synthesis_model
Per-scope synthesis
over all link assessments
critique_synthesis — Phase 3.5b
LLM: steelman critique
per scope
identify_gaps — Phase 3.5c
LLM: gap analysis per scope
Phase 3.5d — Science
dispatch_science_investigations — ROUTER
Send per flagged assumption
K assumptions total
Send x K assumptions
investigate_science_assumption — WORKER
Runs in parallel per assumption


Calls
investigate_science_question()
8 tools available
ASTA gate enforced
asta_soft_cap=5
max_iterations=8
Output: ScienceResult
collect_science_results — JOIN
Aggregate all ScienceResults
Phase 3.7 — Necessity
necessity_check
LLM synthesis_model
Necessity scoring per link per scope
Output: necessity_scores dict
Phase 3.8 — Decision
dispatch_decision_projections — ROUTER
Send per scope S scopes
Send x S scopes
project_scope_decisions — WORKER



Node Operations — Phases 3.5–3.8

Phase 3.5 — Synthesis, Critique, and Gap Analysis

Node	Operation
`synthesize_findings`	LLM call (`synthesis_model`) per scope. Synthesizes all link assessments for a scope into a coherent narrative about the scope's causal validity.
`critique_synthesis`	LLM call (`synthesis_model`) per scope. Applies a steelman critique to the synthesis — considers strongest counter-arguments and disconfirming evidence.
`identify_gaps`	LLM call (`synthesis_model`) per scope. Identifies open scientific and evidential questions that would change the assessment if resolved.

Phase 3.5d — Science Investigation Fan-out

Node	Operation
`dispatch_science_investigations`	Pure router. Emits one `Send()` per flagged scientific assumption (K assumptions identified during gap analysis).
`investigate_science_assumption`	Worker (parallel per assumption). Calls `investigate_science_question()` with 8 tools and a mandatory ASTA gate. The ASTA gate injects a forced Semantic Scholar search if the agent hasn't searched ASTA by the evidence-gathering check. `asta_soft_cap=5` papers, `max_iterations=8`. Produces a `ScienceResult` with evidence quality score.
`collect_science_results`	Aggregates all `ScienceResult` objects via `operator.add`.

Phase 3.7 — Necessity Check

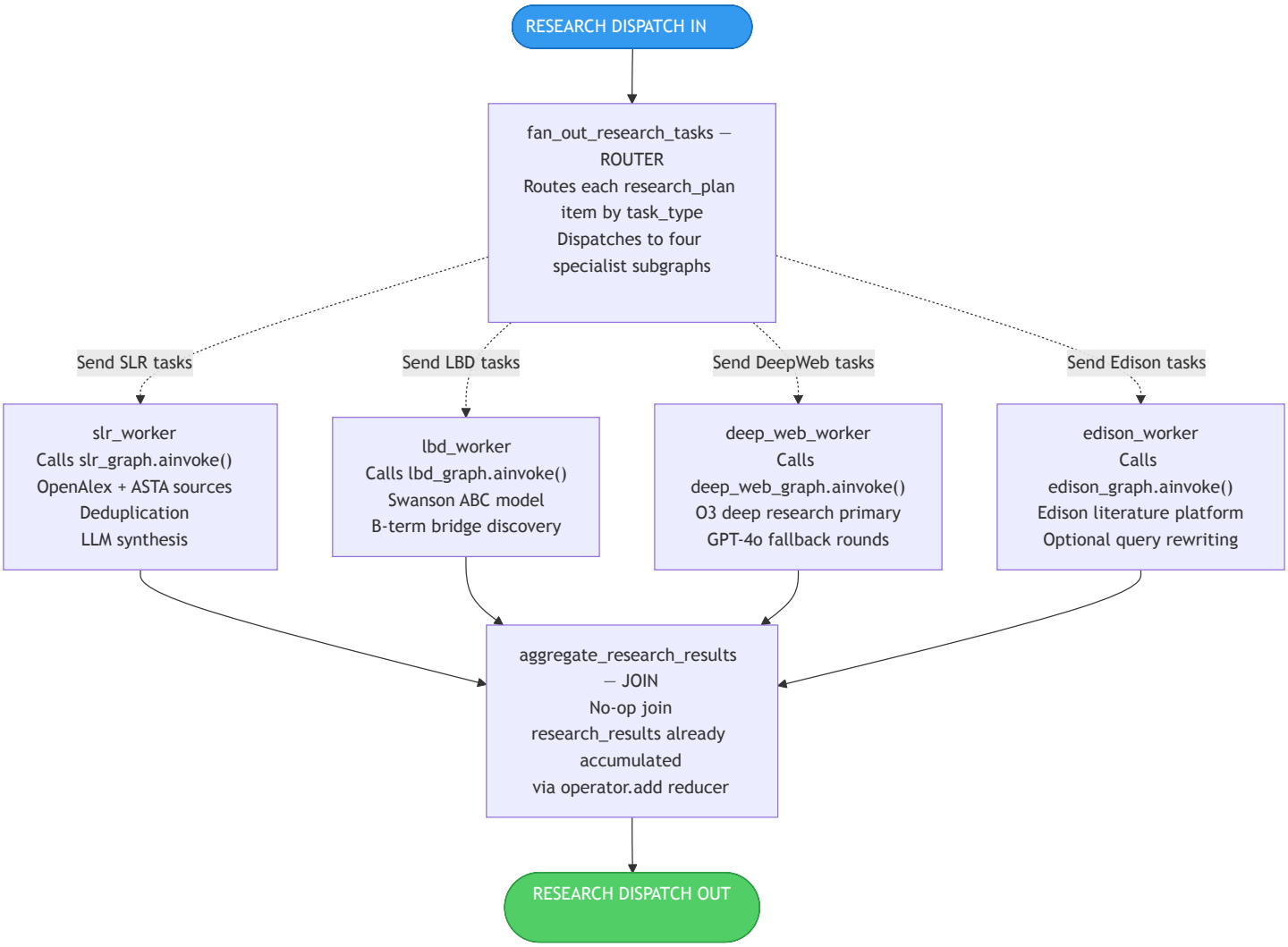
Node	Operation
`necessity_check`	LLM call (`synthesis_model`) per scope. Scores each causal link for necessity — how critical it is to the scope's theory of change. Produces `necessity_scores: dict[link_id → float]` per scope.

Phase 3.8 — Decision Projection Fan-out

Node	Operation
`dispatch_decision_projections`	Pure router. Sends one `Send()` per scope.
`project_scope_decisions`	Worker (parallel per scope). Calls `decision_projection.project_decisions()` which runs four sequential steps: (1) sanitize raw decisions, (2) apply §1a strategic gate to filter low-priority decisions, (3) rank by projected impact, (4) cap the total count. Produces a `ScopeDecision`.
`collect_decisions`	Aggregates all `ScopeDecision` objects. Finalizes the `scope_outputs` dict for each scope.
`clear_fanout_accumulators`	Marker node. Signals that all fan-out data is now embedded in `scope_outputs`. The raw accumulator lists (`evidence_packs`, `link_assessments`, etc.) have served their purpose.

5. Research Dispatch Subgraph

6 nodes · Stage 4 · Routes research plan items to four specialist subgraphs
Figure 5 — Research Dispatch Subgraph



Node Operations

Node	Operation	Specialist subgraph
`fan_out_research_tasks`	Pure router. Inspects `task_type` of each item in `research_plan` and emits `Send()` to the matching worker. All four task types run in parallel.	—
`slr_worker`	Calls `slr_graph.ainvoke()`. The SLR subgraph queries OpenAlex and ASTA, deduplicates papers, and LLM-synthesizes a systematic literature review.	6-node SLR graph
`lbd_worker`	Calls `lbd_graph.ainvoke()`. The LBD subgraph implements Swanson ABC model — discovers B-term concept bridges between seed A-concepts and target C-concepts.	7-node LBD graph
`deep_web_worker`	Calls `deep_web_graph.ainvoke()`. Primary path uses O3 deep research. Falls back to multi-round GPT-4o web search if O3 is unavailable or returns insufficient content.	5-node DeepWeb graph
`edison_worker`	Calls `edison_graph.ainvoke()`. Queries the Edison literature platform, with optional query rewriting via `edison_rewriter.rewrite_query()`.	3-node Edison graph
`aggregate_research_results`	No-op join node. Results are already accumulated in `research_results: Annotated[list, operator.add]` as	—

Node	Operation	Specialist subgraph
	workers complete. This node just provides a convergence point.	

6. State Evolution — Seed to Terminal

How WorkflowState fields accumulate from CLI input through each pipeline phase

Seed State — at START

These fields are set from the CLI invocation. Everything else is `None` or `[]`.

Field	Value	Source
`program`	`"MOCK"` (example)	CLI `--program`
`run_name`	`"run-2026-05-30"`	CLI `--run-name`
`base_dir`	`~/qpr-collections`	CLI `--base-dir`
`focus`	optional string filter	CLI `--focus`
`focus_bows`	optional list of BOW IDs	CLI `--focus-bows`
`research_model`	`claude-sonnet-4-6`	config default
`synthesis_model`	`claude-sonnet-4-6`	config default

After Each Pipeline Phase

Phase / Node	Fields added to WorkflowState
`load_collection`	`doc_list`, `investment_scoring`, `bow_investment_map`, `investment_intelligence`, `chunks_json_path`, `pages_dir`, `ingested_dir`
`precheck`	`precheck_passed`, `precheck_report`
analyze → orientation	`program_context.{theory_of_change, major_bets, stated_priorities, key_timelines, portfolio_health_signals, bow_summaries, initial_concerns}`
analyze → compute_scopes	`scopes: list[ScopeSpec]`
analyze → build_timelines	`scope_timelines: dict[scope_id, ScopeTimeline]` (narratives empty)
analyze → Phase 2.6 narrative fan-out	`scope_timelines[scope_id].narrative` filled per scope in parallel
causal → Phase 3.1 evidence fan-out	`evidence_packs[]` accumulated · `bow_causal_models` · `InvestigationClaim` objects
causal → Phase 3.1.5 BOW enrichment	`scope_outputs[scope_id].bow_context` per scope
causal → Phase 3.2	`scope_outputs[scope_id].consequence_forecasts`
causal → Phase 3.4 link investigation	`link_assessments[]` accumulated · `scope_outputs[scope_id].link_assessments`
causal → Phase 3.5–3.5c	`scope_outputs[scope_id].{synthesis, critique, gaps}` per scope
causal → Phase 3.5d science fan-out	`science_results[]` accumulated · `scope_outputs[scope_id].science_results`
causal → Phase 3.7	`scope_outputs[scope_id].necessity_scores: dict[link_id, float]`

Phase / Node	Fields added to WorkflowState
causal → Phase 3.8 decision fan-out	`scope_decisions[]` accumulated · `scope_outputs[scope_id].decisions: list[ScopeDecision]`
analyze → Phase 3.5 report fan-out	`scope_outputs[scope_id].investment_report` (AI verdict + team score + divergence)
analyze → Phase 3.6 section fan-out	`scope_outputs[scope_id].section_draft: SectionDraft`
analyze → cross_cutting_analysis	`cross_cutting_analysis.{contradictions, shared_dependencies, emergent_decisions, clusters}`
analyze → quality_assessment	`coverage_pct`, `grade`, `confidence_map`
analyze → assemble_report	`final_report_md`, `final_report_md_path`, `bibliography`, `analyst_report`
`deliver`	`excerpts_csv_path`, `final_report_pdf_path`, `current_stage="delivered"`

Reducer Strategy Reference

Field	Reducer	Behavior
`scope_outputs`	`merge_scope_outputs`	Deep-merges by `scope_id` — parallel workers each write different keys to the same scope dict without overwriting each other
`evidence_packs`	`operator.add`	List append — each worker appends one `EvidencePack`; final list has all investments
`link_assessments`	`operator.add`	List append — each link worker appends one `LinkAssessment`
`science_results`	`operator.add`	List append — each science worker appends one `ScienceResult`
`scope_decisions`	`operator.add`	List append — each decision worker appends one `ScopeDecision`
`research_results`	`operator.add`	List append — all four research agent types accumulate here
`errors`	`operator.add`	Accumulates error dicts from all nodes across the full run
`*_traces` (9 fields)	`operator.add`	Accumulates tool call metadata records across the entire run
Agent `messages`	`add_messages`	LangChain built-in deduplication + append for agent conversation history

7. Tool Inventory

Four ToolNode sets, each bound to a specific investigation context

InvestigationToolNode — 12 tools (used in Phase 3.4 link investigation)

Tool	Operation
`search_investment`	Embedding search scoped to a single investment's documents
`search_portfolio`	Embedding search across the entire portfolio collection
`search_bow`	Embedding search scoped to a specific BOW
`search_science`	Semantic Scholar search via ASTA client
`search_policy`	Policy and grey literature search
`search_web`	General web search
`read_document`	Fetch full document text by file_id
`read_section`	Fetch section text by file_id + page range

Tool	Operation
`read_document_summary`	Fetch pre-computed document summary
`get_document_structure`	List sections and page ranges for a document
`list_documents`	List available documents with metadata
`compute`	Code interpreter for numerical analysis

ScienceToolNode — 8 tools (used in Phase 3.5d science investigation)

Tool	Operation
`search_asta`	Semantic Scholar via ASTA — required by ASTA gate before evidence_gathered
`search_bow`	BOW-scoped collection search
`search_science`	Broader academic search
`search_policy`	Policy literature search
`search_web`	Web search
`read_document`	Full document text
`read_section`	Section text by page range
`compute`	Code interpreter

CollectionToolNode — 6 tools (used in Phase 3.1 evidence collection)

Tool	Operation
`search_collection`	Core embedding search against the ingested collection
`read_section`	Section text by file_id + page range
`read_pages`	Full page text
`read_key_docs`	Summary of top K documents
`read_page_image`	PNG page image for multimodal analysis
`get_page_images_for_section`	List images in a document section

NarrationToolNode — 6 tools (used in Phase 2.6 narrative generation)

Tool	Operation
`list_filtered_investments`	List investments by BOW or narrative context
`search_within_scope`	Search scoped to current narrative lens
`read_evidence_pack`	Fetch evidence pack dict for an investment
`read_page_image`	Multimodal evidence retrieval

8. Checkpointing and Resumability

SQLite (default) or Postgres backend · full resume from any node boundary

Checkpoint Strategy

Event	Behavior
Every node exit	State snapshotted to checkpointer via `thread_id = {program}/{run_name}`

Event	Behavior
`load_collection`	Early-return guard: if `doc_list` already populated in checkpointed state, skips disk reads
Worker nodes	Each worker checks disk cache before re-executing LLM work. Result files written to `threads_dir/` keyed by investment/scope/link ID
`collect_timeline_narratives`	Updates `timeline_narratives.json` on disk — narratives survive process restarts
`--interactive` CLI flag	Adds a `interrupt_before=["analyze"]` breakpoint. Run pauses after `precheck`; user can inspect state and send resume value via `main.py resume`
`main.py resume`	Loads last checkpoint for the given `thread_id` and continues from the interrupted node
`main.py status`	Reads checkpoint metadata to report `current_stage` and last completed node

Backend Configuration

Env var	Default	Options
`NQPR_CHECKPOINT_BACKEND`	`sqlite`	`sqlite` · `postgres`
`NQPR_CHECKPOINT_PATH`	`~/.nqpr_checkpoints/checkpoints.db`	Any writable path (SQLite only)
`DATABASE_URL`	—	Postgres connection string
`CHECKPOINT_HUMAN_INTERRUPTS`	`false`	`true` / `false`
`NQPR_SEARCH_BACKEND`	`local`	`local` · `qdrant` · `azure`

Model Configuration Reference

Config key	Default model	Used by
`DEFAULT_RESEARCH_MODEL`	`claude-sonnet-4-6`	Link investigation, science investigation, decision projection
`DEFAULT_SYNTHESIS_MODEL`	`claude-sonnet-4-6`	Report assembly, cross-cutting analysis, synthesis/critique/gaps, necessity check
`DEFAULT_ANALYSIS_MODEL`	`claude-sonnet-4-6`	Orientation, Phase 1
`DEFAULT_FAST_MODEL`	`claude-haiku-4-5-20251001`	Narrative generation, quick structured extractions
`DEFAULT_STRONG_MODEL`	`claude-opus-4-7`	Reserved for high-stakes single calls
`COMPUTE_MODEL`	`gpt-4o-mini`	Code interpreter tool